glibc

- License: LGPL
- C library from the GNU project
- Designed for performance, standards compliance and portability
- Found on all GNU / Linux host systems
- Of course, actively maintained
- By default, quite big for small embedded systems. On armv7hf, version 2.31: `libc`: 1.5 MB, `libm`: 432 KB, source: `https://toolchains.bootlin.com`
- `https://www.gnu.org/software/libc/`

Image: `https://bit.ly/2EzHl6m`

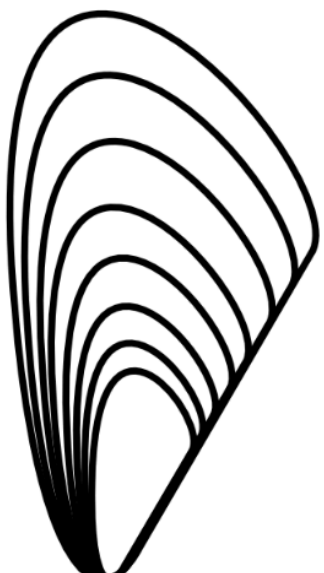
uClibc-ng

- https://uclibc-ng.org/
- A continuation of the old uClibc project, license: LGPL
- Lightweight C library for small embedded systems
  - High configurability: many features can be enabled or disabled through a menuconfig interface.
  - Supports most embedded architectures, including MMU-less ones (ARM Cortex-M, Blackfin, etc.). The only library supporting ARM noMMU.
  - No guaranteed binary compatibility. May need to recompile applications when the library configuration changes.
  - Some features may be implemented later than on glibc (real-time, floating-point operations...)
  - Focus on size (RAM and storage) rather than performance
  - Size on armv7hf, version 1.0.34: `libc`: 712 KB, source: https://toolchains.bootlin.com
- Actively supported, supported by Buildroot but not by Yocto Project.

musl C library

https://www.musl-libc.org/

- A lightweight, fast and simple library for embedded systems
- Created while uClibc's development was stalled
- In particular, great at making small static executables, which can run anywhere, even on a system built from another C library.
- More permissive license (MIT), making it easier to release static executables. We will talk about the requirements of the LGPL license (glibc, uClibc) later.
- Supported by build systems such as Buildroot and Yocto Project.
- Used by the Alpine Linux lightweight distribution (https://www.alpinelinux.org/)
- Size on armv7hf, version 1.2.0: `libc`: 748 KB, source: https://toolchains.bootlin.com



glibc vs uclibc-ng vs musl - small static executables

Let's compile and strip a `hello.c` program **statically** and compare the size

- With musl 1.2.0: **9,084** bytes
- With uclibc-ng 1.0.34: **21,916** bytes.
- With glibc 2.31: **431,140** bytes

Tests run with `gcc 10.0.2` toolchains for `armv7-eabihf` (from `https://toolchains.bootlin.com`)

glibc vs uclibc vs musl - more realistic example

Let's compile and strip BusyBox 1.32.1 **statically** (with the `defconfig` configuration) and compare the size

- With musl 1.2.0: **1,176,744** bytes
- With uclibc-ng 1.0.34: **1,251,080** bytes.
- With glibc 2.31: **1,852,912** bytes

Notes:

- Tests run with `gcc 10.0.2` toolchains for `armv7-eabihf`
- BusyBox is automatically compiled with `-Os` and stripped.
- Compiling with shared libraries will mostly eliminate size differences

Other smaller C libraries

- Several other smaller C libraries have been developed, but none of them have the goal of allowing the compilation of large existing applications
- They can run only relatively simple programs, typically to make very small static executables and run in very small root filesystems.
- Choices:
  - Newlib, https://sourceware.org/newlib/, maintained by Red Hat, used mostly in Cygwin, in bare metal and in small POSIX RTOS.
  - Klibc, https://en.wikipedia.org/wiki/Klibc, from the kernel community, designed to implement small executables for use in an *initramfs* at boot time.

Advice for choosing the C library

- Advice to start developing and debugging your applications with *glibc*, which is the most standard solution, and is best supported by debugging tools (*ltrace* not supported by *musl* in Buildroot, for example).
- Then, when everything works, if you have size constraints, try to compile your app and then the entire filesystem with *uClibc* or *musl*.
- If you run into trouble, it could be because of missing features in the C library.
- In case you wish to make static executables, *musl* will be an easier choice in terms of licensing constraints. The binaries will be smaller too. Note that static executables built with a given C library can be used in a system with a different C library.